RetailDB - Real-time SQL Server Project


```sql
create database retailDB;

use retailDB;

-- Create Customers table
create table Customers
(
    customer_id int primary key identity(1,1),
    FirstName Nvarchar(50),
    LastName Nvarchar(50),
    Email Nvarchar(100),
    Phone Nvarchar(20),S
    Address Nvarchar(200),
    City Nvarchar(50),
    State Nvarchar(50),
    ZipCode Nvarchar(10),
    Country Nvarchar(50),
    CreatedAt datetime default getdate()S
    );

--Create products table
Create table products
(
    product_id int primary key identity(1,1),
    ProductName Nvarchar(100),
    CategoryID int,
    Price Decimal (10,2),
    Stock INT,
    CreateAT Datetime default getdate()
    );

-- Create Categories table
Create table Categories
(
    CategoryID int primary key identity(1,1),
    CategoryName Nvarchar(100),
    Description Nvarchar(255)
    );

-- Create orders tableS
Create table Orders
(
    Order_id int primary key identity(1,1),
    Customer_id int,
    OrderDate datetime default getdate(),
    TotalAmount Decimal(10,2),
    Foreign key (customer_id) references Customers(customer_id)
    );

```

```sql
-- Alter / Rename the column name
-- EXEC sp_rename 'retailDB.dbo.Orders.Customer_id', 'CustomerID', 'COLUMN';

-- Create order_items table
Create table Order_items
(
    OrderItemID int primary key identity(1,1),
    Order_id int,
    Product_id int,
    Quantity int,
    Price Decimal(10,2),
    Foreign key (Order_id) references orders(Order_id),
    Foreign key (Product_id) references products(Product_id)
    );

-- Insert Sample Data into Categories table
insert into Categories (CategoryName, Description)
values
('Electronics', 'Devices and gadgets'),
('Clothing', 'Apparel and accessories'),
('Home & Kitchen', 'Household items and kitchenware');

-- Insert sample data intp Products table
insert into products(ProductName, CategoryID, Price, Stock)
values
('smartphone', 1, 699.99, 50),
('laptop', 1, 999.99, 30),
('T-shirt', 2, 19.99, 100),
('Jeans', 2, 49.99, 80),
('Fiction Novel', 3, 14.99, 200),
('Science Journal', 3, 29.99, 150);

-- Insert sample data into Customers table
INSERT INTO Customers(FirstName, LastName, Email, Phone, Address, City, State, ZipCode, Country)
VALUES
('Sameer', 'Khanna', 'sameer.khanna@example.com', '123-456-7890', '123 Elm St.', 'Springfield',
'IL', '62701', 'USA'),
('Jane', 'Smith', 'jane.smith@example.com', '234-567-8901', '456 Oak St.', 'Madison',
'WI', '53703', 'USA'),
('harshad', 'patel', 'harshad.patel@example.com', '345-678-9012', '789 Dalal St.', 'Mumbai',
'Maharashtra', '41520', 'INDIA');

-- Insert sample data into Orders table
INSERT INTO Orders(Customer_id, OrderDate, TotalAmount)
```

```sql
VALUES
(1, GETDATE(), 719.98),
(2, GETDATE(), 49.99),
(3, GETDATE(), 44.98);

-- Insert sample data into Order_items table
INSERT INTO Order_items(Order_id, Product_id, Quantity, Price)
VALUES
(1, 1, 1, 699.99),
(1, 3, 1, 19.99),
(2, 4, 1,  49.99),
(3, 5, 1, 14.99),
(3, 6, 1, 29.99);


-- Query 1: Retrieve all orders for a specific customer
select o.Order_id, o.OrderDate, o.TotalAmount, oi.Product_id, p.ProductName, oi.Quantity,oi.Price
from Orders o
join Order_items oi on o.Order_id = oi.Order_id
join products p on oi.Product_id = p.product_id
where o.Customer_id = 1; -- Replace with the desired customer ID


-- Query 2: Find the total sales for each product
select p.Product_id, p.ProductName, sum(oi.Quantity * oi.Price) as TotalSales
from products p
join Order_items oi
on oi.product_id = p.product_id
group by p.Product_id, p.ProductName
order by totalsales desc;


-- Query 3: Calculate the average order value
select avg(TotalAmount) as AverageOrderValue from Orders;


-- Query 4: List the top 5 customers by total spending
SELECT Customer_id, FirstName, LastName, TotalSpent, rn
FROM
(SELECT c.Customer_id, c.FirstName, c.LastName, SUM(o.TotalAmount) AS TotalSpent,
ROW_NUMBER() OVER (ORDER BY SUM(o.TotalAmount) DESC) AS rn
FROM Customers c
JOIN Orders o
ON c.customer_id = o.Customer_id
GROUP BY c.customer_id, c.FirstName, c.LastName)
sub WHERE rn <= 5;
```

```sql


-- Query 5: Retrieve the most popular product category
select CategoryID, CategoryName, TotalQuantitysold, rn
from (
select c.CategoryID, c.CategoryName, sum(oi.Quantity) as TotalQuantitysold,
ROW_NUMBER() OVER (ORDER BY sum(oi.Quantity) DESC) as rn
from Order_items oi
join products p
on oi.Product_id = p.product_id
join Categories c
on p.CategoryID = c.CategoryID
group by c.CategoryID, c.CategoryName
) sub where rn = 1;


-- To insert a product with zero stock
insert into products(ProductName, CategoryID, Price, Stock)
values
('Keyboard', 1, 39.99, 0);


-- Query 6: List all products that are out of stock i,e, Stock = 0
select * from products where stock = 0;

select product_id, ProductName, Stock
from products
where stock = 0;

-- with category name
select p.product_id, p.ProductName, c.CategoryName, p.Stock
from products p
join categories c
on p.categoryid = c.CategoryID
where stock = 0;


-- Query 7 : Find Customer who placed order in the last 30 days
select c.customer_id, c.FirstName, c.LastName, o.Orderdate
from Customers c
join Orders o
on c.customer_id = o.customer_id
where o.Orderdate >= DATEADD(day, -30, GETDATE());


-- Query 8: Calculate the total number of orders placed each month
select year(orderDate)  as orderyear,
MONTH(Orderdate) as ordermonth,
```

```sql
count(Order_id) as totalorders
from Orders
Group by year(orderDate), MONTH(Orderdate)
order by orderyear, ordermonth;


-- Query 9: Retrieve the details of the most recent order
select o.Order_id, o.Customer_id, o.OrderDate, o.TotalAmount, c.FirstName, c.LastName
from Orders o
join Customers c
on o.Customer_id = c.customer_id
Order by o.OrderDate DESC;


--Query 10: Find the average price of products in each category

-- FYR: Query 6
-- SELECT p.ProductID, p.ProductName, c.CategoryName, p.Stock
-- FROM Products p JOIN Categories c
-- ON p.CategoryID = c.CategoryID
-- WHERE Stock = 0;
select c.CategoryID, c.CategoryName, AVG(p.Price) as averageprice
from products p
join Categories c
on p.CategoryID = c.CategoryID
group by c.CategoryID, c.CategoryName
order by c.CategoryID,c.CategoryName;


-- Query 11: List customers who have never placed an order
select c.customer_id, c.FirstName, c.LastName, c.Email,c.Phone, c.Address, c.City, c.State, c.ZipCode, c.Country, o.Order_id, o.TotalAmount
from Customers c
join Orders o
on c.Customer_id = o.Customer_id
where order_id is null;


-- Query 12: Retrive the total quantity sold for each product
select p.product_id, p.productname, sum(oi.quantity) as totalquantitysold
from products p
join order_items oi
on p.product_id =  oi.order_id
group by p.product_id, p.productname
order by totalquantitysold Desc;


```

```sql
-- Query 13: Calculate the total revenue generated from each category
select c.CategoryID, c.categoryname, sum(oi.quantity*oi.price) as totalrevenue
from Categories c
join order_items oi
on c.CategoryID = oi.product_id
group by c.CategoryID, c.categoryname
order by totalrevenue desc;


-- Query 14: Find the highest priced product in each category
select p.product_id, p.productname, c.CategoryName, p.Price
from products p
join Categories c
on p.CategoryID = c.CategoryID
where p.price = (
select max(price) from products p2
where p2.categoryid = p.CategoryID
)
order by p.price desc;

-- Query 15: Retrieve orders with a total amount greater than specidic value (e.g, $500)
select o.order_id, c.customer_id, c.FirstName, c.LastName, o.OrderDate, o.TotalAmount
from orders o
join Customers c
on o.Customer_id = c.customer_id
where o.TotalAmount > 49.99
order by o.TotalAmount desc;


-- Query 16: List products along with the number of orders they aapear in
select p.product_id, p.productname, count(oi.order_id) as OrderCount
from products p
join Order_items oi
on p.product_id = oi.Product_id
group by p.product_id, p.productname
order by OrderCount desc;


-- Query 17: Find the top 3 most frequently ordered products
select top 3 p.product_id, p.productname, count(oi.order_id) as OrderCount
from products p
join order_items oi
on p.product_id = oi.Product_id
group by p.product_id, p.productname
```

```sql
order by OrderCount Desc;


-- Query 18: Calculate the total number of customers for each country
select c.Country , count(c.customer_id) as TotalCustomers
from Customers c
group by c.Country
order by TotalCustomers desc;


-- Query 19: Retrieve the list of customers along with their total spending
select c.customer_id, c.FirstName, c.LastName, sum(o.TotalAmount) as TotalSpending
from customers c
join Orders o
on c.customer_id = o.Customer_id
group by c.customer_id, c.FirstName, c.LastName
order by TotalSpending desc;


-- Query 20: List orders with more than a specified number of items (e.g, more than 2 items)
select o.order_id, c.customer_id, c.FirstName, c.LastName, o.OrderDate, count(oi.OrderItemID) as TotalItems
from orders o join order_items oi
on o.Order_id = oi.Order_id
join Customers c
on o.Customer_id = c.customer_id
group by o.Order_id, c.customer_id, c.FirstName, c.LastName, o.OrderDate
having count(oi.orderItemID) > 1
order by TotalItems;


/*
============================
LOG MAINTENANCE
============================
Let's create additional queries that involve updating, deleting, and maintaining logs of these operations
in the RetailDB database.

To automatically log changes in the database, you can use triggers in SQL Server.
Triggers are special types of stored procedures that automatically execute in response
to certain events on a table, such as INSERT, UPDATE, or DELETE.

Here’s how you can create triggers to log INSERT, UPDATE, and DELETE
```

```
  operations
for the tables in the RetailDB.

We'll start by adding a table to keep logs of updates and deletions.

Step 1: Create a Log Table
Step 2: Create Triggers for Each Table

    A. Triggers for Products Table
        -- Trigger for INSERT on Products table
        -- Trigger for UPDATE on Products table
        -- Trigger for DELETE on Products table

    B. Triggers for Customers Table
        -- Trigger for INSERT on Customers table
        -- Trigger for UPDATE on Customers table
        -- Trigger for DELETE on Customers table

    C. Triggers for  Orders Table
        -- Trigger for INSERT on Orders table
        -- Trigger for UPDATE on Orders table
        -- Trigger for DELETE on Orders table

    D. Triggers for Order_items Table
        -- Trigger for INSERT on Order_items table
        -- Trigger for UPDATE on Order_items table
        -- Trigger for DELETE on Order_items table

    E. Triggers for Categories Table
        -- Trigger for INSERT on Categories table
        -- Trigger for UPDATE on Categories table
        -- Trigger for DELETE on Categories table

*/

-- Create a Log Table
CREATE TABLE ChangeLog (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    TableName NVARCHAR(50),
    Operation NVARCHAR(10),
    RecordID INT,
    ChangeDate DATETIME DEFAULT GETDATE(),
    ChangedBy NVARCHAR(100)
);
GO
-- select * from ChangeLog
```

```sql

--A. Triggers for Products Table
-- Trigger for INSERT on Products table
create or alter trigger trg_Insert_products
on products
after insert
as
begin
-- insert a record into ChangeLog table
insert into ChangeLog (TableName, Operation, RecordID, ChangedBy)
select 'products', 'INSERT',inserted.product_id, system_user
from inserted;

-- display a message indicating that the trigger has fired.
print 'INSERT operation logged for products table';
end;
GO

-- Try to insert one record into the Products table
insert into products (ProductName, CategoryID, Price, Stock)
values ('wireless mouse', 1, 4.99, 20),
       ('spiderman multiverse comic', 3, 2.50, 150);
       GO

      -- select * from products;
      -- select * from ChangeLog;

-- Tigger fo UPDATE on product table
create or alter trigger try_update_products
on products
after update
as
begin
    -- insert a record into ChangeLog table
    insert into ChangeLog(TableName, Operation, RecordID, ChangedBy)
    select 'products', 'UPDATE', inserted.product_id, system_user
    from inserted;

    -- diplay a meesage indication that the trigger has fired.
    print 'UPDATE operation logged for products table';
    end;
    GO

-- Try to UPDATE any record from products table
update products set price = price - 100 where product_id = 2;
GO

-- Trigger for DELETE a record from products table
create or alter trigger trg_delete_products
```

```sql
on products
after delete
as
begin

-- insert a record into the changelog table
insert into ChangeLog(TableName, Operation, RecordID, Changedby)
select 'products', 'DELETE', deleted.product_id, SYSTEM_USER
from deleted;

-- Display a message indicating that the trigger has fired.

Print'DELETE operation logged for products table.';
end;
GO

-- Try to delete an existing record to see the effect of Trigger
DELETE FROM products WHERE product_id = 6;

-- Pehle child table se hatao
DELETE FROM Order_items
WHERE Product_id = '6';

-- Ab product delete hoga
DELETE FROM Products
WHERE Product_id = '6';

-- check affected or not from products
select * from products;
GO




-- B. Triggers for Customers Table
-- Trigger for INSERT on Customers table

create or alter trigger trg_Insert_customers
on customers
after insert
as
begin

SET NOCOUNT ON;

-- insert a record into ChangeLog table
insert into ChangeLog (TableName, Operation, RecordID, ChangedBy)
select 'customers', 'INSERT',inserted.customer_id, system_user
from inserted;
```

```sql

-- display a message indicating that the trigger has fired.
print 'INSERT operation logged for Customers table';
end;
GO

-- Try to insert one record into the Customers table
insert into Customers (FirstName, LastName, Email, Phone, Address, City, State, ZipCode, Country)
VALUES
('Virat', 'Kohli', 'virat.kingkohli@example.com', '123-456-7890', 'South Delhi', 'Delhi',
'Delhi', '5456665', 'INDIA');
GO

      -- select * from Customers;
      -- select * from ChangeLog;

-- Tigger fo UPDATE on product table
create or alter trigger try_update_customers
on customers
after update
as
begin

SET NOCOUNT ON;

    -- insert a record into ChangeLog table
    insert into ChangeLog(TableName, Operation, RecordID, ChangedBy)
    select 'customers', 'UPDATE', inserted.customer_id, system_user
    from inserted;

    -- diplay a meesage indication that the trigger has fired.
    print 'UPDATE operation logged for customers table';
    end;
    GO

-- Try to UPDATE any record from products table
update Customers set LastName = 'Chauhan' where customer_id = 4;
GO


-- Trigger for DELETE a record from customers table
create or alter trigger trg_delete_customers
on customers
after delete
as
begin

```

```
SET NOCOUNT ON;

-- insert a record into the changelog table
insert into ChangeLog(TableName, Operation, RecordID, Changedby)
select 'products', 'DELETE', deleted.customer_id, SYSTEM_USER
from deleted;

-- Display a message indicating that the trigger has fired.

Print'DELETE operation logged for Customers table.';
end;
GO

-- Try to delete an existing record to see the effect of Trigger
DELETE FROM Customers WHERE customer_id = 6;
GO



--C. Triggers for  Orders Table
-- Trigger for INSERT on Orders table

create or alter trigger trg_Insert_Orders
on Orders
after insert
as
begin

SET NOCOUNT ON;

-- insert a record into ChangeLog table
insert into ChangeLog (TableName, Operation, RecordID, ChangedBy)
select 'Orders', 'INSERT',inserted.order_id, system_user
from inserted;

-- display a message indicating that the trigger has fired.
print 'INSERT operation logged for Orders table';
end;
GO

-- Trigger for UPDATE on Orders table
create or alter trigger try_update_Orders
on Orders
after update
as
begin

SET NOCOUNT ON;

```

```
    -- insert a record into ChangeLog table
    insert into ChangeLog(TableName, Operation, RecordID, ChangedBy)
    select 'Orders', 'UPDATE', inserted.order_id, system_user
    from inserted;

    -- diplay a meesage indication that the trigger has fired.
    print 'UPDATE operation logged for Orders table';
    end;
    GO

-- Trigger for DELETE on Orders table
create or alter trigger trg_delete_Orders
on Orders
after delete
as
begin

SET NOCOUNT ON;

-- insert a record into the changelog table
insert into ChangeLog(TableName, Operation, RecordID, Changedby)
select 'Orders ', 'DELETE', deleted.order_id, SYSTEM_USER
from deleted;

-- Display a message indicating that the trigger has fired.

Print'DELETE operation logged for Orders  table.';
end;
GO




--D. Triggers for Order_items Table
-- Trigger for INSERT on Order_items table

create or alter trigger trg_Insert_Order_items
on Order_items
after insert
as
begin

SET NOCOUNT ON;

-- insert a record into ChangeLog table
insert into ChangeLog (TableName, Operation, RecordID, ChangedBy)
select 'Order_items', 'INSERT',inserted.OrderItemID, system_user
from inserted;

```

```sql
-- display a message indicating that the trigger has fired.
print 'INSERT operation logged for Order_items table';
end;
GO

-- Trigger for UPDATE on Order_items table
create or alter trigger try_update_Order_items
on Order_items
after update
as
begin

SET NOCOUNT ON;

    -- insert a record into ChangeLog table
    insert into ChangeLog(TableName, Operation, RecordID, ChangedBy)
    select 'Order_items', 'UPDATE', inserted.OrderItemID, system_user
    from inserted;

    -- diplay a meesage indication that the trigger has fired.
    print 'UPDATE operation logged for Order_items table';
    end;
    GO

-- Trigger for DELETE on Orders table
create or alter trigger trg_delete_Order_items
on Order_items
after delete
as
begin

SET NOCOUNT ON;

-- insert a record into the changelog table
insert into ChangeLog(TableName, Operation, RecordID, Changedby)
select 'Order_items', 'DELETE', deleted.OrderItemID, SYSTEM_USER
from deleted;

-- Display a message indicating that the trigger has fired.

Print'DELETE operation logged for Order_items table.';
end;
GO




-- E. Triggers for Categories Table
-- Trigger for INSERT on Categories table
```

```sql
create or alter trigger trg_Insert_Categories
on Categories
after insert
as
begin

SET NOCOUNT ON;

-- insert a record into ChangeLog table
insert into ChangeLog (TableName, Operation, RecordID, ChangedBy)
select 'Categories', 'INSERT',inserted.CategoryID, system_user
from inserted;

-- display a message indicating that the trigger has fired.
print 'INSERT operation logged for Categories table';
end;
GO

-- Trigger for UPDATE on Categories table
create or alter trigger try_update_Categories
on Categories
after update
as
begin

SET NOCOUNT ON;

    -- insert a record into ChangeLog table
    insert into ChangeLog(TableName, Operation, RecordID, ChangedBy)
    select 'Categories', 'UPDATE', inserted.CategoryID, system_user
    from inserted;

    -- diplay a meesage indication that the trigger has fired.
    print 'UPDATE operation logged for Categories table';
    end;
    GO

-- Trigger for DELETE on Categories table
create or alter trigger trg_delete_Categories
on Categories
after delete
as
begin

SET NOCOUNT ON;

-- insert a record into the changelog table
insert into ChangeLog(TableName, Operation, RecordID, Changedby)
select 'Categories', 'DELETE', deleted.CategoryID, SYSTEM_USER
```

```sql
from deleted;

-- Display a message indicating that the trigger has fired.

Print'DELETE operation logged for Categories table.';
end;
GO




/*
================================
Implementing Indexes
================================

Indexes are crucial for optimizing the performance of your SQL Server database,
especially for read-heavy operations like SELECT queries.

Let's create indexes for the RetailDB database to improve query performance.

A. Indexes on Categories Table

1. Clustered Index on CategoryID: Usually created with the primary key.

Clustered Index
Data table ke andar physically sorted hota hai index ke order me.
Matlab table ke rows index ke according arrange ho jate hain.
Sirf 1 clustered index ho sakta hai ek table me.

Example: Phonebook me log name ke order me arranged hote hain → ye clustered index jaisa hai.

Non-Clustered Index
Data table same rehta hai, alag se ek index structure banta hai.
Index me pointer hota hai jo actual data ki location batata hai.
Multiple non-clustered indexes ho sakte hain ek table me.
Example: Book ke end me index page hota hai → word se page number milta hai → ye non-clustered index jaisa hai.

*/

-- Clustered Index on Categories Table (CategoryID)
CREATE CLUSTERED INDEX IDX_Categories_CategoryID
ON Categories(CategoryID);
GO

```

```sql
/*B Indexes on Products Table
 1. Clustered Index on Product_id: This is usually created automatically when the primary key is defined.

 First we need to remove the FK relation between order_items and products table
 to drop the clustered index on product_id column because clustered index is created on primary key column by default.
*/

-- Drop Foreign Key Constraint from order_items table - Product_id
alter table order_items drop constraint FK__Order_ite__Produ__5DCAEF64;

-- Clustered Index on Products Table (ProductID)
CREATE CLUSTERED INDEX IDX_Products_Product_id
ON Products(Product_id);
GO

-- 2. Non-Clustered Index on CategoryID: To speed up queries filtering by CategoryID.
CREATE NONCLUSTERED INDEX IDX_Products_CategoryID
ON Products(CategoryID);
GO

-- 3. Non-Clustered Index on Price: To speed up queries filtering or sorting by Price.
CREATE NONCLUSTERED INDEX IDX_Products_Price
ON Products(Price);
GO

-- Constraints = table ke column par rule jo invalid data ko insert hone se rokta hai.

-- Recreate FK constraint between order_items (Producct_id)
ALTER TABLE Order_items add constraint FK__Order_items_Products
FOREIGN KEY (Product_id) REFERENCES Products(Product_id);
GO




/*
C. Indexes on Orders Table
1. Clustered Index on Order_id: This is usually created automatically when the primary key is defined.
*/
```

```sql

-- Clustered Index on OrderID
CREATE CLUSTERED INDEX IDX_Orders_Order_id
ON Orders(Order_id);
GO

-- Drop Foreign Key Constraint from order_items table - Order_id
Alter Table Order_items drop constraint FK__Order_ite__Order__5CD6CB2B


-- 2. Non - Clustered Index On customer_id: To speed up queries filtering by customer_id.
CREATE NONCLUSTERED INDEX IDX_customers_customers_id
ON CUSTOMERS(customer_id)
GO

-- 3. Non - Clustered Index on OrderDate: To speed up queries filtering or sorting by OrderDate.
CREATE NONCLUSTERED INDEX IDX_ORDERS_OrderDate
on Orders(OrderDate)
GO

-- Recreate FK constraint on Order_items (Order_id column)
ALTER TABLE Order_items add constraint FK__Order_items_Orders
FOREIGN KEY (Order_id) REFERENCES Orders(Order_id);
GO




/*
D. Indexes on OrderItems Table
*/
--1. Clustered Index on OrderItemID: Usually created with the primary key.

-- Clustered Index on OrderItemID
CREATE CLUSTERED INDEX IDX_Orders_order_items
ON Order_items(OrderItemID);
GO

--2. Non-Clustered Index on order_id: To speed up queries filtering by OrderID.
CREATE NONCLUSTERED INDEX IDX_Order_items_OrderID
ON Order_items(Order_id);
GO

--3. Non-Clustered Index on Product_id: To speed up queries filtering by ProductID.
```

```sql
CREATE NONCLUSTERED INDEX IDX_Order_items_Product_id
ON Order_items(Product_id);
GO




/*
E. Indexes on Customers Table
*/

--1. Clustered Index on Customer_id: Usually created with the primary key.
CREATE CLUSTERED INDEX IDX_Customers_Customer_id
on Customers(customer_id);
GO

-- Drop Foreign Key Constraint from Orders table - Customer_id
ALTER TABLE Orders DROP CONSTRAINT FK__Orders__Customer__534D60F1;

--2. Non-Clustered Index on Email: To speed up queries filtering by Email.
CREATE NONCLUSTERED INDEX IDX_Customers_Email
ON Customers(Email);
GO

--3. Non-Clustered Index on Country: To speed up queries filtering by Country.
CREATE NONCLUSTERED INDEX IDX_Customers_Country
ON Customers(Country);
GO

-- Recreate FK constraint on Orders (Customer_id column)
ALTER TABLE Orders ADD CONSTRAINT FK__Orders__Customer_id
FOREIGN KEY (Customer_id) REFERENCES Customers(customer_id);
GO




/*
================================
Implementing Views
================================

Views are virtual tables that represent the result of a query
They can simplify complex queries and enchance security by restricting access to specific date.

```

```sql
*/

-- Query 1:
-- View for products detail: A view combining products detail with category names.
CREATE VIEW VW_ProductsDetail as
select p.product_id, p.productname, p.price, p.stock, c.categoryname
from products p
inner join categories c
on p.CategoryID = c.CategoryID
GO

-- Diplay products detail with category name using view
select * from VW_ProductsDetail;
GO


-- Query 2:
-- View for customer orders: A view to get a summary of orders placed by each customers
CREATE VIEW vw_customerorders
as
select c.customer_id, c.firstname, c.lastname, count(o.order_id)
as
totalorders,

sum(oi.quantity * p.price) as totalamount
from customers c
inner join orders o on c.customer_id = o.customer_id
inner join order_items oi on o.order_id = oi.order_id
inner join products p on oi.product_id = p.product_id
group by c.customer_id, c.firstname, c.lastname ;
GO

  select * from vw_customerorders;
  GO


--  Query 3:
-- view for Recent Orders: A view to display orders placed in the last 30 days
 CREATE view vw_RecentOrders
 as
 select o.order_id, o.OrderDate, c.customer_id, c.FirstName, c.LastName,
 sum(oi.quantity * oi.price) as OrderAmount
 from customers c
 inner join orders o on c.customer_id = o.Customer_id
 inner join Order_items oi on o.Order_id = oi.Order_id
 Group BY o.Order_id, o.OrderDate, c.customer_id,c.FirstName, c.LastName;
```

```sql
 GO

 select * from vw_RecentOrders;
 GO


-- Query 4:
-- Retrieve All products with category Names
-- Using the vw_ProductDetails view to get a list of all products along wtih their category names

 Select * from VW_ProductsDetail;
 Go


-- Query 5:
-- Retrieve Products within a specific price range
-- Using the vw_productDetails where price between 10 and 500;

  select * from VW_ProductsDetail
  where price
  between 10 and 500;
  GO


-- Query 6:
-- Count the number of products in each category
-- Using the vw_productDetails view to count the number of products in each category

  select categoryname, count(product_id) as productcount
  from VW_ProductsDetail
  group by categoryname;
  GO


-- Query 7:
-- Retrieve Customers with more than 1 orders
-- using the vw_customerorders view to find customers who have placed more than 1 orders.

   select * from vw_customerorders
   where totalorders > 1;
   GO


-- Query 8:
-- Retrieve the total amount spent by each customers
-- Using the vw_customerOrders view to get the total amount spent by
```

```
each customer.

 select customer_id, firstname, lastname, totalamount
 from vw_customerorders
 order by totalamount DESC;
 GO


-- Query 9:
-- Retrieve Recent orders above a certain amount
-- using the vw_RecentOrders view to find recent orders where the total amount is greater than 500.

  select * from vw_RecentOrders
  where OrderAmount > 500;
  GO


-- Query 10:
-- Retrieve the latest order fr each customer
-- Using the vw_RecentOrderss view to find the latest order placed by each customer.

  select  customer_id, FirstName, LastName, OrderDate as LatestOrderDate
  from vw_RecentOrders
  ORDER BY OrderDate DESC;
  GO


-- Query 11:
-- Retrieve products in a specific category
-- Using the vw_productDetails view to get all products in a specific category such as electronics.

   select * from VW_ProductsDetail
   where categoryname = 'Electronics';
   GO


-- Query 12:
-- Retreive TotalSales for each category
-- using the vw_procuctDetails and vw_cusotmerorders view to calculate the total sales for each category.

   select categoryname, sum(totalamount) as TotalSales
   from VW_ProductsDetail pd
   join order_items oi on pd.product_id = oi.product_id
   join orders o on oi.order_id = o.order_id
```

```sql
     join customers c on o.customer_id = c.customer_id
     group by categoryname
     order by TotalSales desc;
     GO

    -- SELECT pd.categoryname, SUM(oi.Quantity * p.Price) AS TotalSales
    -- FROM Order_Items oi
    -- INNER JOIN Products p ON oi.product_id = p.product_id
    -- INNER JOIN vw_ProductsDetail pd ON p.product_id = pd.product_id
    -- GROUP BY pd.categoryname
    -- ORDER BY TotalSales DESC;
    -- GO


-- Query 13:
-- Retrieve customers orders with product Details
-- Using the vw_customerOrders and vw_ProductsDetail view to get customer orders along with the details of the the products ordered.

    SELECT co.customer_id, co.FirstName, co.LastName, o.Order_id, o.OrderDate, pd.ProductName, oi.Quantity, oi.Price
    from vw_customerorders co
    join orders o on co.customer_id = o.Customer_id
    inner join order_items oi on o.Order_id = oi.Order_id
    inner join VW_ProductsDetail pd on oi.product_id = pd.product_id
    inner join vw_customerorders co2 on co.customer_id = co2.customer_id
    order by o.OrderDate desc;
    GO

-- Query 14:
-- Retrieve top 5 customers by total spending
-- using the vw_customerorders view to find the top 5 customers based on their total spending. `

    SELECT top 5 customer_id, FirstName, LastName, totalamount as TotalSpending
    from vw_customerorders
    order by TotalSpending desc;
    GO


-- Query 15:
-- Retrieve Products with low stock
-- Using the vw_productsdetail view to find products with stock below a certain threshold, such as 10 units.

   select * from VW_ProductsDetail
   where stock < 30;
   GO
```

```sql

-- Query 16:
-- Retrieve Orders Placed in the last 7 Days
-- Using the vw_RecentOrders view to find orders that were placed in the last 7 days.

    select * from vw_RecentOrders;

    select * from vw_RecentOrders
    where OrderDate >= DATEADD(day, -7, GETDATE());
    GO

    -- select top 7 * from vw_RecentOrders
    -- Order by OrderDate desc;
    -- GO

-- Query 17:
-- Retrieve products sold in the last month
-- Using the vw_RecentOrders view to find products sold in last month.

  select p.product_id, p.productname, sum(oi.quantity) as TotalQuantitySold
  from vw_RecentOrders ro
  inner join order_items oi on ro.Order_id = oi.Order_id
  inner join products p on oi.product_id = p.product_id
  where ro.OrderDate >= DATEADD(month, -1, GETDATE())
  group by p.product_id, p.productname
  order by TotalQuantitySold desc;
  GO


/*
============================================================
Implementing Security / Role-Based Access Control (RBAC)
============================================================

Role-Based Access Control (RBAC) is a security model
that restricts access to resources based on the roles of individual

TO Manage access Control in sql server we'll need to use a combination of sql server's security features
Such as
Login,
Users,
Roles,
and Permissions.

*/

```

```sql
-- Step by Step guide how to implement RBAC in SQL Server for the RetailDB database:


-- 1. Create Logins: Logins are used to authenticate users at the SQL Server level.
-- Create a login with sql server authentication

   Create login SalesUser with password = 'NewStrong@123';
   GO


-- 2. Create Users : Users are created within a specific database and are associated with logins.
-- Create a user in DB for the sql server login
   use retailDB;
   GO

   Create user SalesUser for login SalesUser;
   GO


-- 3. Create Roles: Roles are created to group permissions together.
-- You can create custom roles based on the needs of your application,helps simplify permission management.
-- Create roles in the database

   CREATE ROLE SalesRole;
   CREATE ROLE MarkingRole;
   CREATE ROLE InventoryRole;
   GO


-- 4. Assign User to Role
-- Add user to the appropriate role
-- Add users to Role
   Exec sp_addrolemember 'SalesRole', 'SalesUser';
   GO
  -- sp = in-build store procedure like addrolemember etc


-- 5. Grant Permissions: Grant the necessary permissions to the roles based on the access Requirements
-- GRANT SELECT permission on the Customers Table to the SalesRole
   GRANT SELECT ON Customers TO SalesRole;
   GO

-- GRANT INSERT permission on the orders table to the SalesRole
   GRANT INSERT on Orders to SalesRole;
```

```sql
   GO

-- GRANT UPDATE permission on the orders table to the SalesRole
   GRANT UPDATE on Orders to SalesRole;
   GO

 -- GRANT SELECT permission on the products table to the SalesRole
   GRANT SELECT on products to SalesRole;
   GO

/*

   Select * from customers;
   DELETE from Customers; -- This will give an error because SalesUser doesn't have DELETE permission on Customers table.

   Select * from Orders;
   DELETE FROM Orders;
   INSERT INTO Orders(Customer_id, OrderDate, TotalAmount)
   VALUES (1, GETDATE(), 600);

   SELECT * FROM Products;
   DELETE FROM Products;

*/


-- 6.Revoke Permission (if Needed)
-- If you need to revoke a permission, you can use the REVOKE statement.
-- For example, to revoke the INSERT permission on the Orders table from the SalesRole:
   REVOKE INSERT on Orders from SalesRole;
   GO


-- 7. View Effective Permission
-- You can view the effective permissions for a user or role using the in-build sys.database_permissions catalog view.
   SELECT * FROM fn_my_permissions('null', 'DATABASE');
   GO

/*
===================
Summary
===================
    1. Create Logins: Authenticate users at the SQL Server level.
    2. Create Users: Create users in the database for the logins.
    3. Create Roles: Group users with similar permissions.
    4. Assign Users to Roles: Add users to appropriate roles.
```

```sql
    5. Grant Permissions: Grant necessary permissions to roles.
    6. Revoke Permissions: Revoke permissions if needed.
    7. View Effective Permissions: Check the effective permissions for users.
*/


/*
    Here are 20 different scenarios for access control in SQL Server.
    These scenarios cover various roles and permissions that can be assigned to users
    in the `RetailDB` database.
*/

-- Scenario 1: Read-Only Access to all tables
CREATE ROLE ReadOnlyRole
GRANT SELECT ON SCHEMA::dbo TO ReadOnlyRole;

-- Scenario 2: Data Entry Clerk (Insert Only on Orders and Order_items)
CREATE ROLE DataEntryClerkRole
GRANT INSERT ON Orders TO DataEntryClerkRole;
GRANT INSERT ON Order_items TO DataEntryClerkRole;

-- Scenario 3: Product Manager (Full Access to Products and Categories)
CREATE ROLE ProductManagerRole
GRANT SELECT, INSERT, UPDATE, DELETE ON Products TO ProductManagerRole;
GRANT SELECT, INSERT, UPDATE, DELETE ON Categories TO ProductManagerRole;

-- Scenario 4: Order processor (Read and Update Access to Orders)
CREATE ROLE OrderProcessorRole
GRANT SELECT, UPDATE ON Orders TO OrderProcessorRole;
GO

-- Scenaro 5: Customer Suppport (Read Access to Customers and Orders)
CREATE ROLE CustomerSupportRole
GRANT SELECT ON Customers TO CustomerSupportRole;
GO

-- Scenario 6: Marketing Analyst (Read Access to Orders and Order_items)
CREATE ROLE MarketingAnalystRole
GRANT SELECT ON Orders TO MarketingAnalystRole;
GO

-- Scenario 7: Sales Analyst (Read Access to Orders, Order_items, and Products)
CREATE ROLE SalesAnalystRole
GRANT SELECT ON Orders TO SalesAnalystRole;
GO

```

```sql
-- Scenario 8: Inventory Manager (Full Access to Products)
CREATE ROLE InventoryManagerRole
GRANT SELECT, INSERT, UPDATE, DELETE ON Products TO InventoryManagerRole;
GO

-- Scenario 9: Finance Analyst (Read And Update Orders)
CREATE ROLE FinanceAnalystRole
GRANT SELECT, UPDATE ON Orders TO FinanceAnalystRole;
GO

-- Scenario 10: Data Backup Operator (Backup Database)
CREATE ROLE BackupOperatorRole
GRANT BACKUP DATABASE TO BackupOperatorRole;
GO

-- Scenario 11: Database Developer (Full Access to Schema Objects)
CREATE ROLE DatabaseDeveloperRole;
GRANT CREATE TABLE TO DatabaseDeveloperRole;
GRANT ALTER ON SCHEMA::dbo TO DatabaseDeveloperRole;
GRANT CONTROL ON SCHEMA::dbo TO DatabaseDeveloperRole;

-- Scenario 12: Restricate Read Accesse(Read only specific Columns)
CREATE ROLE RestrictedReadRole;
GRANT SELECT ON Customers(FirstName, LastName, Email)
TO RestrictedReadRole;

-- Scenario 13: Reporting User (Read Access to Views Only)
CREATE ROLE ReportingUserRole;
GRANT SELECT ON VW_ProductsDetail TO ReportingUserRole;
GRANT SELECT ON vw_customerorders TO ReportingUserRole;

-- Scenario 14: Temporary Access (Time-Bound Access)
-- GRANT Access
CREATE ROLE TemporaryAccessRole;
GRANT SELECT ON SCHEMA::dbo TO TemporaryAccessRole;

-- Revoke Access after the specified period
REVOKE SELECT ON SCHEMA::dbo FROM TemporaryAccessRole;

-- Scenario 15: External Auditor (Read Access to All Tables with no data change)
CREATE ROLE AuditorRole;
GRANT SELECT ON SCHEMA::dbo TO AuditorRole;
DENY INSERT, UPDATE, DELETE ON SCHEMA::dbo TO AuditorRole;

-- Scenario 16: Application Role (Access Based on Application)
CREATE APPLICATION ROLE AppRole WITH PASSWORD = 'NewStrong@123';
GRANT SELECT, INSERT, UPDATE ON Orders TO AppRole;
GO
```

```sql

-- Scenario 17: Role-Based Access (RBAC) for Multiple Roles
-- Combine Roles for a user
CREATE ROLE CombineRole;
EXEC sp_addrolemember 'SalesRole', 'CombineRole';
EXEC sp_addrolemember 'MarkingRole', 'CombineRole';

-- Scenario 18: Sensitive Data Access (Column Level Permissions)
CREATE ROLE SensitiveDataRole;
GRANT SELECT ON Customers(Email, Phone) TO SensitiveDataRole;
GO

-- Scenario 19: Developer Role (Full Access to Development Database)
CREATE ROLE DeveloperRole;
GRANT CONTROL ON DATABASE::RetailDB TO DeveloperRole;
GO

-- Scenario 20: Security Administrator (Full Access to Security Management)
CREATE ROLE SecurityAdminRole;

GRANT ALTER ANY USER TO SecurityAdminRole;
GRANT ALTER ANY ROLE TO SecurityAdminRole;
```